

Lab Copy – Django Setup & Simple App (blog_users)

Name: Sagar Maity

Roll No.: 120

Date: 30.11.25

Experiment: Install Django, create project & app, templates, static files, model, migrate, display data.

Objective

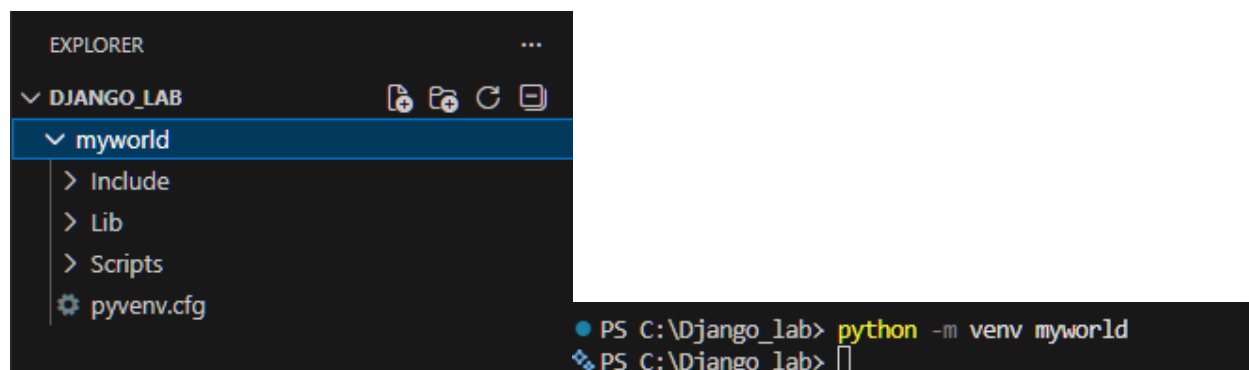
Install Django in a virtual environment, create a project and app, render templates and static files, create a model, add data, and display it.

Environment / Tools

- OS: Windows.
- Python ≥ 3.x, pip, venv.

1. Create virtual environment

- ```
o cd C:\path\to\workfolder
o python -m venv myworld
```



Explorer or dir showing myworld folder.

## 2. Activate environment

- **PowerShell:**

```
o myworld\Scripts\Activate.ps1
```

After activation prompt becomes: (myworld) C:\path\to\workfolder\>

```
(myworld) PS C:\Django_lab>
```

Terminal after activation.

**PowerShell note (if blocked):**

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser  
(only if allowed).

## 3. Install Django

(myworld)

```
o python -m pip install Django
```

## 4. Check Django version

(myworld)

```
o django-admin --version
```

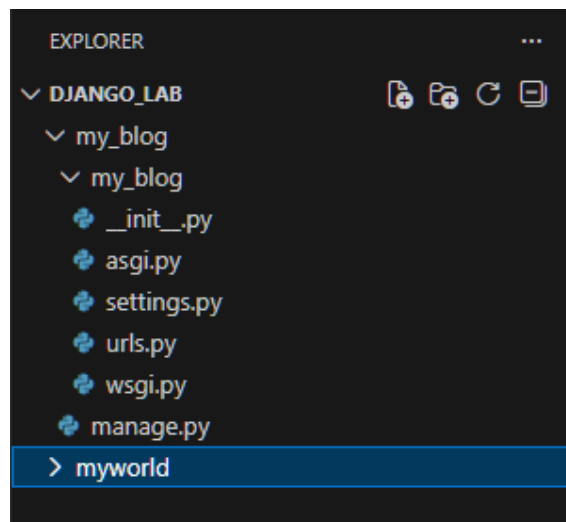
```
(myworld) PS C:\Django_lab> django-admin --version
5.2.8
```

Django version in terminal.

## 5. Create Django project

(myworld)

```
o django-admin startproject my_blog
o cd my_blog
```

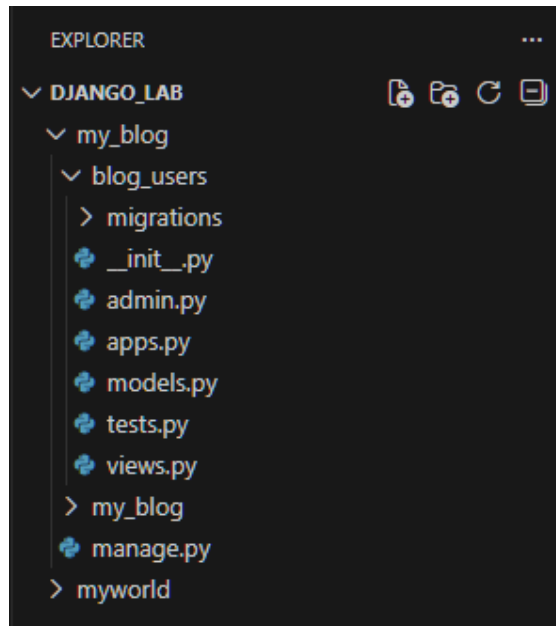


Folder tree showing manage.py and my\_blog/.

## 6. Create app `blog_users`

(myworld)

```
o python manage.py startapp blog_users
```



**blog\_users/ folder structure (with migrations/, views.py, etc.).**

## 7. Run server (test)

(myworld)

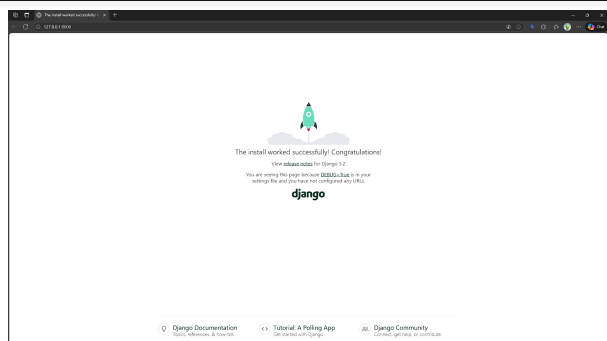
```
o python manage.py runserver
```

```
(myworld) PS C:\Django_lab\my_blog> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
November 30, 2025 - 19:02:19
Django version 5.2.8, using settings 'my_blog.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[30/Nov/2025 19:02:24] "GET / HTTP/1.1" 200 12068
Not Found: /favicon.ico
[30/Nov/2025 19:02:24] "GET /favicon.ico HTTP/1.1" 404 2209
```



Open: `http://127.0.0.1:8000/` in browser.

**Browser showing Django default page or terminal showing server started.**

Stop server with Ctrl+C to continue.

## Files to create / edit — exact code

### A — blog\_users/views.py

```
from django.http import HttpResponse
from django.template import loader
from .models import Bloguser

def blog_users(request):
 return HttpResponse("Hello world!")

def members(request):
 myusers = Bloguser.objects.all().values()
 template = loader.get_template('all_users.html')
 context = {'myusers': myusers}
 return HttpResponse(template.render(context, request))
```

*(One view for test, one to show DB data.)*

### B — create blog\_users/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
 path('blog-users/', views.blog_users, name='blog-users'),
 path('all-users/', views.members, name='all-users'),
]
```

### C — edit project urls my\_blog/urls.py

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
 path('', include('blog_users.urls')),
 path('admin/', admin.site.urls),
]
```

*(Registers app routes.)*

## D – blog\_users/templates/myfirst.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="utf-8">
 <title>My First Django</title>

 <!-- Bootstrap CSS -->
 <link
 href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
 rel="stylesheet"
 integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMHjY6hW+ALEwIH"
 crossorigin="anonymous">

 <!-- Your static CSS -->
 <link rel="stylesheet" href="{% static 'myfirst.css' %}">
</head>

<body class="bg-light">

 <div class="container py-5">
 <div class="card shadow-lg p-4">
 <h1 class="text-center text-primary">Hello World!</h1>
 <p class="text-center fs-4">Welcome to my first Django project!</p>
 </div>
 </div>

 <!-- Bootstrap JavaScript -->
 <script

src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
 integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDz0xhy9GkcIdslK1eN7N6jIeHz"
 crossorigin="anonymous"></script>

</body>
</html>
```

## E — blog\_users/templates/all\_users.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Blog Users</title>

 <!-- Bootstrap CSS -->
 <link

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
 rel="stylesheet"
 integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
 crossorigin="anonymous">
</head>

<body class="bg-light">

<div class="container py-5">
 <div class="card shadow-lg">
 <div class="card-header bg-primary text-white text-center">
 <h2>Blog Users</h2>
 </div>

 <div class="card-body">
 <ul class="list-group">
 {% for x in myusers %}
 <li class="list-group-item fs-5">
 {{ x.firstname }} {{ x.lastname }}

 {% empty %}
 <li class="list-group-item text-danger">No users found.
 {% endfor %}

 </div>
 </div>
 </div>

 <!-- Bootstrap JS -->
 <script

src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
 integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDz0xhy9GkcIdslK1eN7N6jIeHz"
 crossorigin="anonymous">
</script>

</body>
</html>
```

## F – blog\_users/static/myfirst.css

```
body {
 background-color: lightblue;
 font-family: verdana;
}
```

## G – blog\_users/models.py

```
from django.db import models

class Bloguser(models.Model):
 firstname = models.CharField(max_length=255)
 lastname = models.CharField(max_length=255)

 def __str__(self):
 return f"{self.firstname} {self.lastname}"
```

## H – register app in my\_blog/settings.py (add in INSTALLED\_APPS)

```
INSTALLED_APPS = [
 'django.contrib.admin',
 'django.contrib.auth',
 'django.contrib.contenttypes',
 'django.contrib.sessions',
 'django.contrib.messages',
 'django.contrib.staticfiles',
 'blog_users', # <-- add this
]
```

(Then save.)

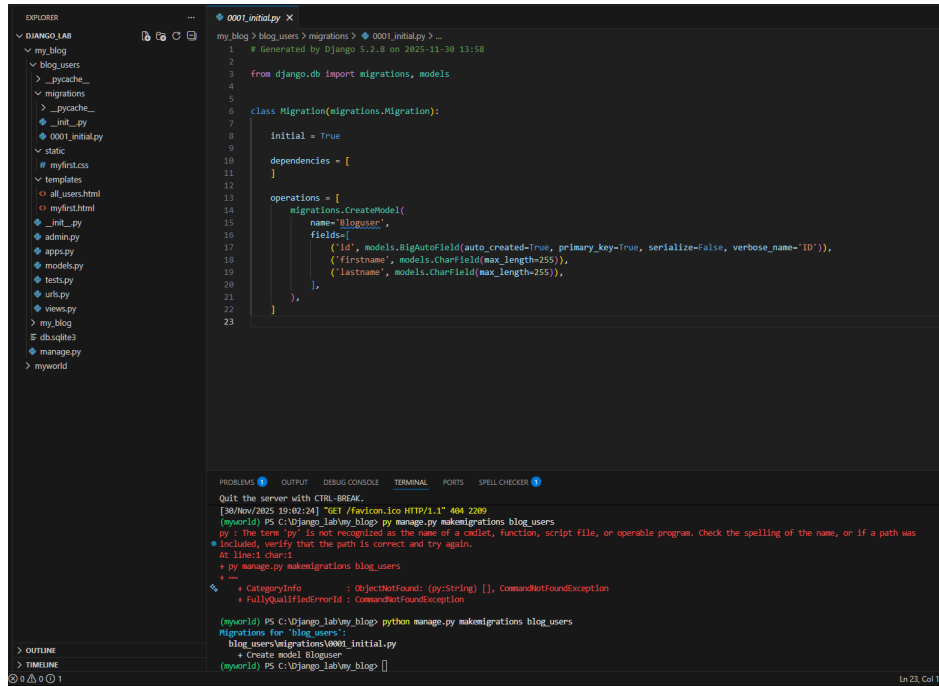
---

# Database: migrations & add data

## 1. Make migrations:

(myworld)

```
python manage.py makemigrations blog_users
```



```
0001_initial.py
1 # Generated by Django 5.2.8 on 2025-11-30 13:58
2
3 from django.db import migrations, models
4
5
6 class Migration(migrations.Migration):
7
8 initial = True
9
10 dependencies = [
11]
12
13 operations = [
14 migrations.CreateModel(
15 name='Bloguser',
16 fields=[
17 ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
18 ('first_name', models.CharField(max_length=255)),
19 ('last_name', models.CharField(max_length=255)),
20],
21),
22]
```

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
Quit the server with CTRL-BREAK.
[30/Nov/2025 19:02:24] "GET /favicon.ico HTTP/1.1" 404 2209
(myworld) PS C:\Django_lab\my_blog> python manage.py makemigrations blog_users
py: the term 'py' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was
included, verify that the path is correct and try again.
At line:1 char:1
+ py manage.py makemigrations blog_users
+ ~~~
+ CategoryInfo : ObjectNotFound: (py:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

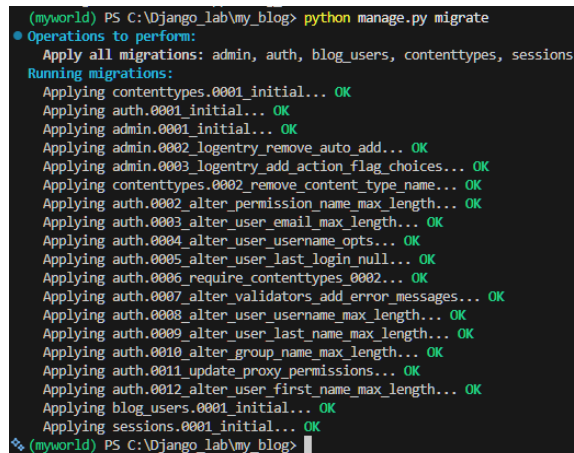
(myworld) PS C:\Django_lab\my_blog> python manage.py makemigrations blog_users
Migrations for 'blog_users':
 blog_users/migrations/0001_initial.py
 + Create model Bloguser
(myworld) PS C:\Django_lab\my_blog>
```

Terminal with 0001\_initial.py created.

## 2. Apply migrations:

(myworld)

```
python manage.py migrate
```



```
(myworld) PS C:\Django_lab\my_blog> python manage.py migrate
• Operations to perform:
 Apply all migrations: admin, auth, blog_users, contenttypes, sessions
Running migrations:
 Applying contenttypes.0001_initial... OK
 Applying auth.0001_initial... OK
 Applying admin.0001_initial... OK
 Applying admin.0002_logentry_remove_auto_add... OK
 Applying admin.0003_logentry_add_action_flag_choices... OK
 Applying contenttypes.0002_remove_content_type_name... OK
 Applying auth.0002_alter_permission_name_max_length... OK
 Applying auth.0003_alter_user_email_max_length... OK
 Applying auth.0004_alter_user_username_opts... OK
 Applying auth.0005_alter_user_last_login_null... OK
 Applying auth.0006_require_contenttypes_0002... OK
 Applying auth.0007_alter_validators_add_error_messages... OK
 Applying auth.0008_alter_user_username_max_length... OK
 Applying auth.0009_alter_user_last_name_max_length... OK
 Applying auth.0010_alter_group_name_max_length... OK
 Applying auth.0011_update_proxy_permissions... OK
 Applying auth.0012_alter_user_first_name_max_length... OK
 Applying blog_users.0001_initial... OK
 Applying sessions.0001_initial... OK
(myworld) PS C:\Django_lab\my_blog>
```

terminal showing migrations applied.



### 3. Add records in Django shell:

(myworld)

```
o python manage.py shell
```

Inside shell:

```
from blog_users.models import Bloguser
bloguser = Bloguser(firstname='Amit', lastname='Roy')
bloguser.save()
```

```
b1 = Bloguser(firstname='Ritesh', lastname='Roy')
b2 = Bloguser(firstname='Soham', lastname='Sarkar')
b3 = Bloguser(firstname='Sourav', lastname='Das')
for b in (b1, b2, b3): b.save()
```

```
Bloguser.objects.all().values()
```

Exit Django Shell:

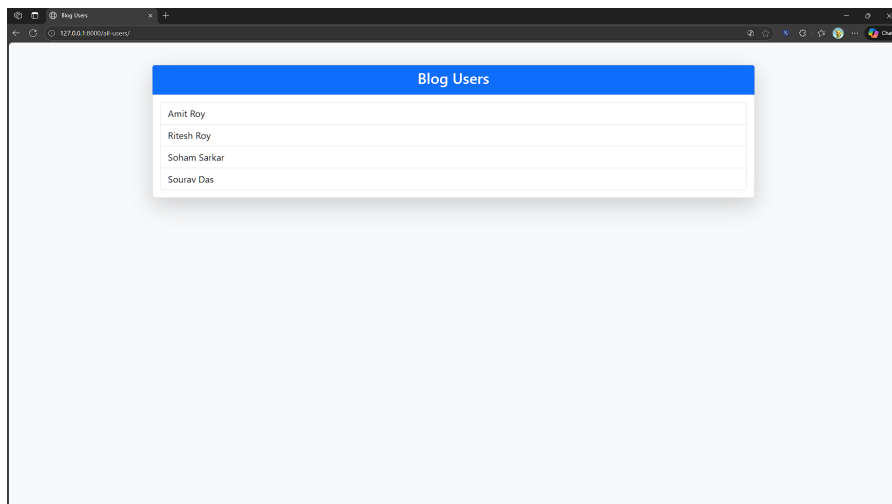
```
o exit()
```

### 4. Run server and view list:

(myworld)

```
o python manage.py runserver
```

Open: <http://127.0.0.1:8000/all-users/> — you should see the users.



**Browser showing Blog Users list.**

# Add data from Admin

## 1. Register the model in admin (one-time)

Edit blog\_users/admin.py:

```
from django.contrib import admin
from .models import Bloguser

admin.site.register(Bloguser)
```

## 2. Create a superuser to access admin (one-time)

In terminal :

```
python manage.py createsuperuser
```

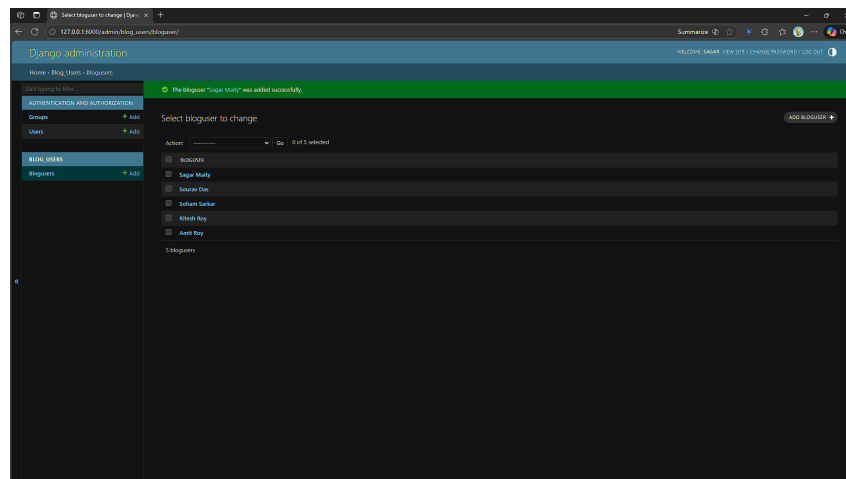
Enter username, email, password.

## 3. Run the server and open admin

```
python manage.py runserver
```

Open browser: <http://127.0.0.1:8000/admin/>

- Login with the superuser credentials.
- Click **Blogusers** → **Add Bloguser** → fill firstname and lastname → **Save**.



After saving.